



Vue.js HTTP 통신 & 컴포넌트 간 통신

Vue 3를 활용한 API 통신 실전 가이드

 Axios로 외부 데이터 불러오기

 비동기 로딩/에러 처리 패턴

 axios-mock-adapter로 테스트 환경

 mitt로 컴포넌트 간 이벤트 통신

1. Axios로 외부 데이터 불러오기 (개념/설치/핵심 메서드)

↔ HTTP 통신이란?

브라우저에서 서버로 요청을 보내고, JSON 데이터를 받아오는 작업
프론트엔드와 백엔드 사이의 주요 소통 방식

? 왜 Axios인가?

간결한 API, 브라우저 호환성 우수
Vue에는 기본 HTTP 클라이언트가 없어 외부 라이브러리 필요

↓ 설치 방법

NPM을 통한 설치:

```
npm install axios
```

컴포넌트에서 임포트:

```
import axios from 'axios'
```

</> 핵심 메서드

axios.get(url) - 서버에서 데이터 가져오기

```
const response = await axios.get('/api/users')
```

axios.post(url, data) - 서버로 데이터 보내기

```
await axios.post('/api/users', { name: '홍길동' })
```

🧪 실습 API

오늘 실습에 활용할 무료 테스트 API: JSONPlaceholder

```
https://jsonplaceholder.typicode.com/users
```

1-2. 기본 실습: UserListBasic.vue

🎯 목표

JSONPlaceholder API에서 사용자 목록을 가져와 화면에 렌더링

🗄️ 상태 관리

users - ref 배열로 유저 데이터 저장

🕒 생명주기

onMounted - 컴포넌트가 마운트될 때 API 호출

⋮ 렌더링

v-for - 배열을 순회하며 각 항목 렌더링
:key - 고유 식별자로 렌더링 최적화

```
src/components/UserListBasic.vue

<li v-for="user in users" :key="user.id">
  {{ user.name }} ({{ user.email }})
</li>
</ul>
</section>
</template>

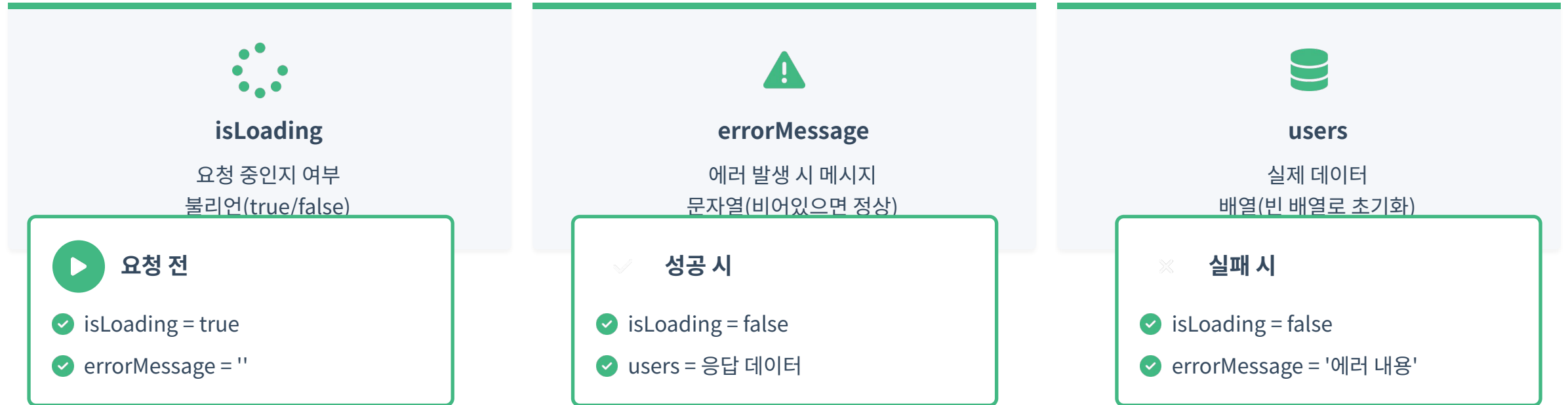
<script setup>
import { ref, onMounted } from 'vue'
import axios from 'axios'

// 1) 가져온 유저 데이터를 저장할 상태
const users = ref([])

// 2) 컴포넌트가 화면에 처음 나타날 때 API 호출
onMounted(async () => {
  const response = await axios.get('https://jsonplaceholder.typicode.com/users')
  users.value = response.data
})
</script>
```

2. 비동기 데이터 처리와 로딩 UI (개념)

네트워크 요청은 항상 지연이나 실패 가능성이 있습니다. 사용자 경험을 위해 3가지 상태를 관리하고 UI를 적절히 분기해야 합니다.



템플릿 분기 처리

```
<!-- 로딩 상태 -->
<p v-if="isLoading">로딩 중입니다...</p>

<!-- 에러 상태 -->
<p v-else-if="errorMessage" style="color: red">
  {{ errorMessage }}
</p>

<!-- 정상 데이터 -->
<ul v-else>
  <li v-for="user in users" :key="user.id">
    {{ user.name }} ({{ user.email }})
  </li>
</ul>
```

2-2. 로딩 UI 실습: UserListWithLoading.vue

🕒 패턴 구현

try/catch/finally로 비동기 상태를 안전하게 관리

📦 3가지 상태 관리

isLoading - 요청 진행 중인지 여부

errorMessage - 오류 메시지 저장

users - 성공 시 데이터 저장

🔗 UI 분기 처리

v-if - 로딩 중 표시

v-else-if - 에러 메시지 표시

v-else - 데이터 목록 표시

🔄 재시도 기능

"다시 불러오기" 버튼으로 API 호출 반복 가능

```
src/components/UserListWithLoading.vue

const users = ref([])
const isLoading = ref(false)
const errorMessage = ref('')
const wait = (ms) => new Promise(resolve => setTimeout(resolve, ms));
const fetchUsers = async () => {
  isLoading.value = true
  errorMessage.value = ''

  try {
    // ⌚ 3초 대기
    await wait(3000)

    const response = await axios.get('https://jsonplaceholder.typicode.com/users')
    users.value = response.data
  } catch (error) {
    console.error(error)
    errorMessage.value = '유저 목록을 불러오는 데 실패했습니다.'
  } finally {
    isLoading.value = false
  }
}
```

3. Axios Mock Adapter로 테스트 환경 (개념/장점)

백엔드 API가 완성되지 않았거나 접근이 불가능한 환경에서도 프론트엔드 개발을 진행할 수 있는 가짜 서버 응답 시뮬레이션 도구입니다.



Mock이란?

실제 서버가 아닌, 미리 정의된 응답을 반환하는 가짜 API 서버입니다.

```
npm install axios-mock-adapter
```

HTTP 요청을 가로채서 네트워크 통신 없이 정의된 응답을 즉시 반환합니다.



주요 장점

- ✓ 백엔드 의존도 제거 - API 개발과 상관없이 프론트엔드 개발 진행 가능
- ✓ 일관된 테스트 환경 - 항상 동일한 응답으로 안정적인 테스트 가능
- ✓ 에러 케이스 시뮬레이션 - 400, 500 등 다양한 에러 상황 테스트 용이



구조 설계

 axiosInstance.js

공통 Axios 설정 (baseUrl, timeout 등)



 mock.js

요청 URL별 가짜 응답 정의

지연 시간, 상태 코드 설정

 main.js



개발 환경에서만 Mock 활성화

3-2. Mock 설정 코드 (axiosInstance, mock.js, main.js)

Mock Adapter를 활용한 테스트 환경은 3개의 핵심 파일로 구성됩니다. 각 파일의 역할과 코드를 살펴보겠습니다.

src/api/axiosInstance.js

```
// src/api/axiosInstance.js
import axios from 'axios'
const api = axios.create({
  baseURL: '/', // 실제 서버 연결 시 변경
  timeout: 3000,
})
export default api
```

i 역할

- 모든 API 요청의 기본 설정을 관리하는 Axios 인스턴스
- baseURL: API 서버 주소 (실제 서버 연결 시 수정)
- timeout: 요청 제한 시간 (3초)

src/api/mock.js

```
// src/api/mock.js
import MockAdapter from 'axios-mock-adapter'
import api from './axiosInstance'
// mock 활성화 함수
export function setupMock() {
  const mock = new MockAdapter(api, {
    delayResponse: 800 // 0.8초 딜레이
  })
  // GET /api/todos 응답
  mock.onGet('/api/todos').reply(200, [
```

i 역할

- 특정 URL에 대한 가짜 응답 정의
- delayResponse: 실제 API처럼 지연 시간 설정
- onGet/onPost: HTTP 메서드별 응답 설정
- reply(상태코드, 데이터): 반환할 응답 형식 지정

3-3. Mock 설정 코드 (axiosInstance, mock.js, main.js)

src/main.js

```
import { createApp } from 'vue'
import App from './App.vue'
import './style.css'
// 📌 mock 설정 import
import { setupMock } from './api/mock'
// 개발 단계에서만 호출한다고 가정
if (import.meta.env.DEV) {
  setupMock()
}
createApp(App).mount('#app')
```

💡 핵심 포인트

- 개발 환경에서만 Mock을 활성화: `if (import.meta.env.DEV) { setupMock() }`
- 실제 서버가 완성되면 이 부분만 주석 처리하거나 제거하면 됨
- 환경변수를 통한 조건부 활성화로 배포 환경에서는 실제 서버 API를 사용

3-4. Mock 실습: TodoListMock.vue

☰ 상태 관리

todos - Todo 배열 저장

isLoading - 로딩 상태

errorMessage - 에러 메시지

newTitle - 새 할 일 입력값

↗ 데이터 요청

fetchTodos() - API에서 할 일 목록 조회

addTodo() - 새 할 일 추가

✍ UI 구성

로딩/에러 상태 처리

완료된 할 일 취소선 표시(textDecoration)

새 할 일 입력 폼과 추가 버튼

💡 Mock 장점

실제 서버 없이 API 흐름 테스트 가능

일관된 응답으로 UI 개발 집중

src/components/TodoListMock.vue

```
try {
  const res = await api.get('/api/todos')
  todos.value = res.data
} catch (e) {
  console.error(e)
  errorMessage.value = 'TODO 목록을 불러오는 데 실패했습니다.'
} finally {
  isLoading.value = false
}

async function addTodo() {
  if (!newTitle.value.trim()) return

  try {
    const res = await api.post('/api/todos', { title: newTitle.value })
    todos.value.push(res.data)
    newTitle.value = ''
  } catch (e) {
    console.error(e)
    alert('TODO 추가에 실패했습니다.')
  }
}
```

4. mitt로 컴포넌트 간 통신 (이벤트 vs 상태 관리)

⚠ Vue 2의 Event Bus 문제점

- 전역 Vue 인스턴스를 이상하게 활용하는 안티패턴 (new Vue() 남용)
- 이벤트 추적이 어렵고 디버깅이 복잡함
- 규모가 커질수록 유지보수 어려움

💡 선택 가이드라인

- 데이터를 공유하고 싶으면 Pinia 사용 (상태 중심)
- 이벤트 신호만 보내고 싶으면 mitt 사용 (알림 중심)

🔊 mitt (이벤트 버스)

- ✓ 초경량 이벤트 라이브러리
on, off, emit 3가지 API만 기억하면 됨
- ✓ 느슨한 결합
발신자와 수신자가 서로를 알 필요 없음
- ✓ 일회성, 알림성 이벤트에 적합
 - 토스트 메시지 표시
 - 모달 열기/닫기
 - 전역 로딩 상태 알림

```
// eventBus.js
import mitt from 'mitt'
const emitter = mitt()
export default emitter
// 발신: bus.emit('event', data)
// 수신: bus.on('event', handler)
```

🗄 Pinia (상태 관리)

- ✓ 중앙 집중식 상태 관리
애플리케이션의 전역 상태를 하나의 저장소에서 관리
- ✓ 데이터 중심 구조
상태(state), 게터(getters), 액션(actions)으로 구조화
- ✓ 영구적 데이터에 적합
 - 로그인 유저 정보
 - 장바구니
 - 다크모드/테마 설정

```
// store.js
import { defineStore } from 'pinia'
export const useUserStore = defineStore('user', {
  state: () => ({ name: '', isLoggedIn: false }),
  actions: { login() { /* ... */ } }
})
```

4-2. mitt 실습: 전역 로딩 바

npm install mitt

mitt를 활용해 컴포넌트 간 통신으로 전역 로딩 상태를 관리하는 예제입니다. API 요청 시작/종료 시 이벤트를 발생시키고, 이를 수신해 전역 로딩 바를 표시합니다.

eventBus.js

```
// src/eventBus.js
import mitt from 'mitt'
const emitter = mitt()
export default emitter
```

UserListWithMitt.vue

```
// 발신 컴포넌트
import { ref, onMounted } from 'vue'
import axios from 'axios'
import bus from '../eventBus'
const users = ref([])
const errorMessage = ref('')
async function fetchUsers() {
  errorMessage.value = ''
  bus.emit('loading', true)
  try {
    const res = await
    axios.get('https://jsonplaceholder.typicode.com/users')
    users.value = res.data
  } catch (e) {
    errorMessage.value = '유저 목록 로드 실패'
  } finally {
    bus.emit('loading', false)
  }
}
```

App.vue

```
// 수신 컴포넌트
import { ref, onMounted, onBeforeUnmount } from 'vue'
import bus from '../eventBus'
const globalLoading = ref(false)
function handleLoading(isLoading) {
  globalLoading.value = isLoading
}
onMounted(() => {
  bus.on('loading', handleLoading)
})
onBeforeUnmount(() => {
  bus.off('loading', handleLoading)
})
// 템플릿
<div v-if="globalLoading" class="loading-bar">
  전역 로딩 중...
</div>
```

💡 핵심 포인트

- 단 3줄의 코드로 이벤트 버스 구현
- mitt() 함수로 이벤트 관리 객체 생성
- 앱 전역에서 import하여 사용 가능

🚀 이벤트 발신

- API 요청 시작 시: bus.emit('loading', true)
- API 요청 완료 시: bus.emit('loading', false)
- 이벤트 이름과 데이터를 전달

🎧 이벤트 수신

- 마운트 시: bus.on('loading', 핸들러) 등록
- 언마운트 전: bus.off('loading', 핸들러) 해제
- 메모리 누수 방지를 위해 해제 필수!

정리 및 다음 단계

학습 내용 정리

Axios로 HTTP 통신

기본 메서드(get/post)와 onMounted에서 데이터 로드하는 패턴 학습

비동기 UI 상태 관리

isLoading과 errorMessage로 사용자 경험을 보장하는 패턴 습득

Mock Adapter로 테스트 환경 구성

백엔드 의존도 최소화, 재현 가능한 API 테스트 환경 구축

mitt로 컴포넌트 간 통신

이벤트 버스 패턴과 Pinia와의 차이점, 선택 가이드라인 이해